

50 Kotlin LiveData Examples

With Detailed Comments & Practice Questions

A Comprehensive Guide to Android LiveData

Example 1: Basic LiveData Declaration

```
class MyViewModel : ViewModel() {  
    // Private MutableLiveData - can be modified within ViewModel  
    private val _counter = MutableLiveData<Int>()  
  
    // Public immutable LiveData - exposed to UI layer  
    // This prevents external classes from modifying the data  
    val counter: LiveData<Int> = _counter  
  
    init {  
        // Set initial value to 0  
        _counter.value = 0  
    }  
}
```

Questions:

1. Why do we expose LiveData as immutable (LiveData) instead of MutableLiveData?
2. What is the initial value of counter?
3. What is the purpose of the underscore prefix in _counter?

Code Challenges:

1. Write code to create a LiveData with initial value 'Hello'
2. How would you change the counter value to 10?
3. Create a LiveData for tracking loading state

Example 2: Observing LiveData in Activity

```
class MainActivity : AppCompatActivity() {
    private lateinit var viewModel: MyViewModel

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        // Get or create ViewModel instance
        // ViewModelProvider ensures same instance across config changes
        viewModel = ViewModelProvider(this).get(MyViewModel::class.java)

        // Observe LiveData - 'this' is the LifecycleOwner
        // Observer automatically stops when Activity is destroyed
        viewModel.counter.observe(this) { count ->
            // This lambda is called whenever counter value changes
            // Only called when Activity is in STARTED or RESUMED state
            textView.text = count.toString()
        }
    }
}
```

Questions:

1. What is the lifecycle owner passed to observe()?
2. When will the observer stop receiving updates?
3. What happens if the Activity is in the background?

Code Challenges:

1. Write an observer for LiveData that updates a TextView
2. How would you observe the same LiveData in two different TextViews?
3. Write code to observe LiveData and display count

Example 3: Updating LiveData Value

```
class CounterViewModel : ViewModel() {
    // Initialize with default value of 0
    private val _count = MutableLiveData<Int>(0)
    val count: LiveData<Int> = _count

    fun increment() {
        // Elvis operator (?:) provides default value if current value is null
        // This is safe even if _count.value somehow becomes null
        _count.value = (_count.value ?: 0) + 1
    }

    fun decrement() {
        _count.value = (_count.value ?: 0) - 1
    }
}
```

Questions:

1. Why do we use `?: 0` when updating the value?
2. Can `increment()` be called from a background thread?
3. What's the difference between `value` and `postValue()`?

Code Challenges:

1. Write a `reset()` function to set count back to 0
2. Write a function `addValue(n: Int)` to add any number
3. Write a function `multiplyBy(n: Int)` to multiply count

Example 4: Using `postValue()` for Background Threads

```
class DataViewModel : ViewModel() {
    private val _data = MutableLiveData<String>()
    val data: LiveData<String> = _data

    fun loadData() {
        // viewModelScope - coroutine scope tied to ViewModel lifecycle
        viewModelScope.launch(Dispatchers.IO) {
            // This runs on IO thread (background)
            val result = fetchDataFromNetwork()

            // postValue() is thread-safe and posts to main thread
            // Use postValue() from background threads
            _data.postValue(result)
        }
    }

    private fun fetchDataFromNetwork(): String {
        // Simulate network delay
        Thread.sleep(2000)
        return "Data loaded"
    }
}
```

Questions:

1. Why use `postValue()` instead of `value` here?
2. What thread does `postValue()` execute on?
3. What is `viewModelScope`?

Code Challenges:

1. Write a function that loads data and sets loading state
2. Modify `loadData()` to handle errors with `try-catch`
3. Write code to cancel the loading operation

Example 5: LiveData with Data Class

```
// Data class - immutable data holder
data class User(
    val id: Int,
    val name: String,
    val email: String
)

class UserViewModel : ViewModel() {
    private val _user = MutableLiveData<User>()
    val user: LiveData<User> = _user

    fun loadUser() {
        // Create and set user object
        _user.value = User(1, "John Doe", "john@example.com")
    }

    fun updateUserName(newName: String) {
        // Create new User with updated name (immutable pattern)
        _user.value?.let { currentUser ->
            _user.value = currentUser.copy(name = newName)
        }
    }
}
```

Questions:

1. What type of data does this LiveData hold?
2. How would you access the user's name in an observer?
3. Is the User data class mutable or immutable?

Code Challenges:

1. Write an observer that displays user name and email
2. Write a function updateUserEmail(email: String)
3. Create a LiveData that can be null

Example 6: LiveData with List

```
class ItemsViewModel : ViewModel() {
    private val _items = MutableLiveData<List<String>>()
    val items: LiveData<List<String>> = _items

    init {
        // Initialize with empty list
        _items.value = emptyList()
    }

    fun addItem(item: String) {
        // Get current list or empty list if null
        val currentList = _items.value ?: emptyList()

        // Create NEW list with added item
        // LiveData only detects changes when a new object is assigned
        _items.value = currentList + item
    }

    fun removeItem(item: String) {
        val currentList = _items.value ?: emptyList()
        // Create new list without the item
        _items.value = currentList.filter { it != item }
    }
}
```

Questions:

1. Why do we create a new list instead of modifying the existing one?
2. What would happen if we used `mutableListOf()` and modified it directly?
3. How does LiveData detect changes in the list?

Code Challenges:

1. Write a function `clearAllItems()`
2. Write a function `addMultipleItems(items: List)`
3. Write a function `getItemCount()` that returns LiveData

Example 7: Transformations.map()

```
class NameViewModel : ViewModel() {
    private val _firstName = MutableLiveData<String>()

    // Transformations.map() creates a NEW LiveData that transforms values
    // Every time _firstName changes, displayName updates automatically
    val displayName: LiveData<String> = Transformations.map(_firstName) { name ->
        // Transformation logic - runs every time source changes
        "Hello, $name!"
    }

    fun setFirstName(name: String) {
        _firstName.value = name
    }
}

// Usage in Activity:
// viewModel.displayName.observe(this) { greeting ->
//     textView.text = greeting // Will show "Hello, John!"
// }
```

Questions:

1. What is the purpose of Transformations.map()?
2. When does the transformation execute?
3. Is displayName a new LiveData object?

Code Challenges:

1. Create a LiveData that shows double the value of input
2. Transform LiveData to show character count
3. Create a LiveData to uppercase transformation

Example 8: Transformations.switchMap()

```
class UserRepository {
    // Simulates fetching user from database or API
    fun getUser(userId: Int): LiveData<User> {
        val result = MutableLiveData<User>()
        // In real app, this would query database
        result.value = User(userId, "User $userId", "user$userId@example.com")
        return result
    }
}

class UserViewModel : ViewModel() {
    private val _userId = MutableLiveData<Int>()
    private val repository = UserRepository()

    // switchMap() switches to a different LiveData based on trigger value
    // When _userId changes, it stops observing old LiveData
    // and starts observing the new one returned by repository.getUser()
    val user: LiveData<User> = Transformations.switchMap(_userId) { id ->
        // This returns a NEW LiveData for each user ID
        repository.getUser(id)
    }

    fun loadUser(id: Int) {
        // Changing userId triggers switchMap to fetch new user
        _userId.value = id
    }
}
```

Questions:

1. What's the difference between map() and switchMap()?
2. When would you use switchMap() over map()?
3. What happens to the old LiveData when userId changes?

Code Challenges:

1. Write switchMap() to get posts for a user ID
2. Create a search function using switchMap()
3. Use switchMap() to load user profile picture

Example 9: MediatorLiveData - Combining Two LiveData

```
class CombinedViewModel : ViewModel() {
    private val _firstName = MutableLiveData<String>()
    private val _lastName = MutableLiveData<String>()

    // MediatorLiveData can observe multiple LiveData sources
    // and react to any of them changing
    val fullName = MediatorLiveData<String>().apply {
        // Add first source - triggered when firstName changes
        addSource(_firstName) { firstName ->
            value = "$firstName ${_lastName.value ?: ""}"
        }

        // Add second source - triggered when lastName changes
        addSource(_lastName) { lastName ->
            value = "${_firstName.value ?: ""} $lastName"
        }
    }

    fun setFirstName(name: String) { _firstName.value = name }
    fun setLastName(name: String) { _lastName.value = name }
}
```

Questions:

1. What is MediatorLiveData used for?
2. How many sources does this MediatorLiveData observe?
3. What happens when only firstName changes?

Code Challenges:

1. Combine LiveData score and level to show total
2. Create MediatorLiveData to show if both fields are filled
3. Combine temperature in C and F LiveData objects

Example 10: observeForever() - Non-Lifecycle Aware

```
class DataManager {
    private val viewModel = MyViewModel()

    // Store observer reference to remove it later
    private val observer = Observer<String> { data ->
        println("Data: $data")
    }

    fun startObserving() {
        // observeForever() does NOT respect lifecycle
        // Observer will receive updates even when not needed
        // ALWAYS requires manual removal!
        viewModel.data.observeForever(observer)
    }

    fun stopObserving() {
        // CRITICAL: Must manually remove observer to prevent memory leaks
        viewModel.data.removeObserver(observer)
    }
}

// WARNING: If you forget to call stopObserving(),
// you will have a memory leak!
```

Questions:

1. Why is observeForever() dangerous?
2. What must you always do when using observeForever()?
3. When would you use observeForever() instead of observe()?

Code Challenges:

1. Write safe observeForever with cleanup in a class
2. Create a function that logs all LiveData changes
3. Show how to use observeForever in a Service

Example 11: LiveData with Nullable Types

```
class NullableViewModel : ViewModel() {
    // String? means the value can be null
    private val _message = MutableLiveData<String?>()
    val message: LiveData<String?> = _message

    fun setMessage(msg: String?) {
        _message.value = msg
    }

    fun clearMessage() {
        // Set to null to indicate no message
        _message.value = null
    }
}

// Observing nullable LiveData:
// viewModel.message.observe(this) { msg ->
//     if (msg != null) {
//         textView.text = msg
//         textView.visibility = View.VISIBLE
//     } else {
//         textView.visibility = View.GONE
//     }
// }
```

Questions:

1. Why would you use a nullable LiveData type?
2. How should observers handle null values?
3. What's the difference between null and an empty string?

Code Challenges:

1. Create LiveData for optional user data
2. Write observer that handles null by showing placeholder
3. Create a function hasMessage() returning LiveData

Example 12: SingleLiveEvent Pattern

```
class SingleLiveEvent<T> : MutableLiveData<T>() {
    // AtomicBoolean ensures thread-safe flag checking
    private val pending = AtomicBoolean(false)

    override fun observe(owner: LifecycleOwner, observer: Observer<in T>) {
        super.observe(owner) { t ->
            // Only notify if event is pending
            // compareAndSet atomically checks and sets the value
            if (pending.compareAndSet(true, false)) {
                observer.onChange(t)
            }
        }
    }

    override fun setValue(t: T?) {
        // Mark event as pending before setting value
        pending.set(true)
        super.setValue(t)
    }
}

class EventViewModel : ViewModel() {
    // Use for one-time events like navigation, snackbar messages
    private val _navigateEvent = SingleLiveEvent<Unit>()
    val navigateEvent: LiveData<Unit> = _navigateEvent

    fun navigate() {
        _navigateEvent.value = Unit
    }
}
```

Questions:

1. What problem does SingleLiveEvent solve?
2. Why is AtomicBoolean used?
3. When would you use this pattern?

Code Challenges:

1. Create SingleLiveEvent for showing toast messages
2. Write SingleLiveEvent for navigation with data
3. Create event for showing error dialogs

Example 13: LiveData in Fragment

```
class MyFragment : Fragment() {
    // by viewModels() delegate creates/retrieves ViewModel
    private val viewModel: MyViewModel by viewModels()

    override fun onCreateView(view: View, savedInstanceState: Bundle?) {
        super.onCreateView(view, savedInstanceState)

        // IMPORTANT: Use viewLifecycleOwner, NOT 'this'
        // viewLifecycleOwner is tied to Fragment's View lifecycle
        // Fragment's view can be destroyed/recreated while Fragment lives
        viewModel.data.observe(viewLifecycleOwner) { data ->
            // Safe to access views here
            binding.textView.text = data
        }
    }
}

// If you use 'this' as lifecycle owner:
// - Observer will outlive the view
// - Memory leak and crashes when accessing destroyed views
```

Questions:

1. Why use viewLifecycleOwner instead of this?
2. What's the difference between Fragment's lifecycle and View's lifecycle?
3. What happens when the Fragment's view is destroyed but Fragment is not?

Code Challenges:

1. Write Fragment code to observe user data
2. Create a Fragment observing multiple LiveData
3. Show how to share ViewModel between Fragments

Example 14: LiveData with Sealed Class for UI State

```
// Sealed class represents all possible UI states
sealed class UiState<out T> {
    // Loading state - show progress indicator
    object Loading : UiState<Nothing>()

    // Success state - show data
    data class Success<T>(val data: T) : UiState<T>()

    // Error state - show error message
    data class Error(val message: String) : UiState<Nothing>()
}

class DataViewModel : ViewModel() {
    private val _uiState = MutableLiveData<UiState<List<String>>>()
    val uiState: LiveData<UiState<List<String>>> = _uiState

    fun loadData() {
        // Set loading state
        _uiState.value = UiState.Loading

        viewModelScope.launch {
            try {
                val data = fetchData()
                // Set success state with data
                _uiState.value = UiState.Success(data)
            } catch (e: Exception) {
                // Set error state with message
                _uiState.value = UiState.Error(e.message ?: "Unknown error")
            }
        }
    }

    private suspend fun fetchData(): List<String> {
        delay(2000)
        return listOf("Item 1", "Item 2")
    }
}

// Observing in Activity:
// viewModel.uiState.observe(this) { state ->
//     when (state) {
//         is UiState.Loading -> showLoading()
//         is UiState.Success -> showData(state.data)
//         is UiState.Error -> showError(state.message)
//     }
// }
```

Questions:

1. What are the benefits of using a sealed class for UI state?
2. How would you handle each state in the UI?
3. Why is this better than multiple LiveData objects?

Code Challenges:

1. Add an Empty state to the sealed class
2. Create UiState for authentication (LoggedIn, LoggedOut, Error)
3. Write observer code that handles all three states

Example 15: Debouncing User Input with LiveData

```
class SearchViewModel : ViewModel() {
    private val _query = MutableLiveData<String>()

    // switchMap creates new LiveData for each query change
    val searchResults: LiveData<List<String>> = _query.switchMap { query ->
        // LiveData builder creates a LiveData with coroutine support
        LiveData {
            // Wait 300ms before searching (debounce)
            // If user types again, this coroutine is cancelled
            delay(300)

            // emit() sets the LiveData value
            emit(performSearch(query))
        }
    }

    fun onQueryChanged(query: String) {
        // Each keystroke triggers this
        _query.value = query
    }

    private suspend fun performSearch(query: String): List<String> {
        // Simulate API call
        delay(500)
        return listOf("Result 1 for $query", "Result 2 for $query")
    }
}

// Without debouncing: searches on EVERY keystroke
// With debouncing: only searches after 300ms of no typing
```

Questions:

1. What is debouncing and why is it useful?
2. What does the `delay(300)` accomplish?
3. What is `LiveData` builder function?

Code Challenges:

1. Change debounce delay to 500ms
2. Add minimum query length check before searching
3. Create debounced email validation

Example 16: LiveData with Repository Pattern

```
// Repository interface - abstraction for data layer
interface Repository {
    fun getData(): LiveData<List<String>>
}

// Concrete implementation
class DataRepository : Repository {
    private val _data = MutableLiveData<List<String>>()

    override fun getData(): LiveData<List<String>> = _data

    fun refreshData() {
        // In real app: fetch from database/network
        _data.value = listOf("Item 1", "Item 2", "Item 3")
    }
}

// ViewModel depends on interface, not implementation
// This enables easy testing with mock repositories
class DataViewModel(private val repository: Repository) : ViewModel() {
    // ViewModel just exposes data from repository
    val data: LiveData<List<String>> = repository.getData()
}

// Benefits:
// - Separation of concerns
// - Testability (can inject mock repository)
// - Single source of truth
```

Questions:

1. What is the Repository pattern?
2. Why separate the repository from the ViewModel?
3. How would you inject the repository?

Code Challenges:

1. Create UserRepository with getUser(id: Int)
2. Write a refreshData() function in ViewModel
3. Create a test using a mock repository

Example 17: LiveData with Coroutines

```
class CoroutineViewModel : ViewModel() {
    private val _result = MutableLiveData<String>()
    val result: LiveData<String> = _result

    fun loadData() {
        // viewModelScope is a CoroutineScope tied to ViewModel
        // Automatically cancelled when ViewModel is cleared
        viewModelScope.launch {
            // This runs on Main thread by default
            _result.value = "Loading..."

            // Simulate async operation
            delay(2000)

            _result.value = "Data loaded successfully"
        }
    }

    fun loadDataFromNetwork() {
        viewModelScope.launch {
            // Switch to IO dispatcher for network call
            val data = withContext(Dispatchers.IO) {
                // Network call here
                "Network data"
            }

            // Back on Main thread automatically
            _result.value = data
        }
    }
}
```

Questions:

1. What is viewModelScope?
2. What happens when the ViewModel is cleared?
3. Can you use setValue() inside a coroutine?

Code Challenges:

1. Write a function that loads data with error handling
2. Create parallel loading of two data sources
3. Implement retry logic using coroutines

Example 18: LiveData Coroutine Builder

```
class ApiViewModel : ViewModel() {
    // LiveData {} builder creates LiveData with coroutine support
    // Coroutine starts when LiveData becomes active (has observers)
    // Coroutine cancels when LiveData becomes inactive
    val data: LiveData<String> = LiveData {
        // Show loading state first
        emit("Loading...")

        // Switch to IO thread for API call
        val result = withContext(Dispatchers.IO) {
            // Simulate API call
            delay(2000)
            "API Response"
        }

        // Emit final result
        emit(result)
    }

    // With timeout
    val dataWithTimeout: LiveData<String> = LiveData(timeoutInMs = 5000) {
        emit("Loading...")
        // If this takes >5s, coroutine is cancelled
        val result = fetchData()
        emit(result)
    }

    private suspend fun fetchData(): String {
        delay(1000)
        return "Data"
    }
}
```

Questions:

1. What does the LiveData builder do?
2. When does the coroutine start executing?
3. How is this different from using MutableLiveData?

Code Challenges:

1. Create LiveData that fetches user profile
2. Add error handling to LiveData builder
3. Create LiveData that emits multiple values over time

Example 19: Combining Multiple LiveData Sources

```
class StatsViewModel : ViewModel() {
    private val _score = MutableLiveData<Int>(0)
    private val _level = MutableLiveData<Int>(1)

    // MediatorLiveData combines multiple sources
    val stats = MediatorLiveData<String>().apply {
        // Local function to update combined value
        fun update() {
            val score = _score.value ?: 0
            val level = _level.value ?: 1
            value = "Score: $score, Level: $level"
        }

        // Call update() whenever score changes
        addSource(_score) { update() }

        // Call update() whenever level changes
        addSource(_level) { update() }
    }

    fun addScore(points: Int) {
        _score.value = (_score.value ?: 0) + points

        // Level up every 100 points
        val currentScore = _score.value ?: 0
        _level.value = (currentScore / 100) + 1
    }
}
```

Questions:

1. Why define update() as a local function?
2. How many times will stats update if both sources change?
3. What's an alternative to MediatorLiveData for combining sources?

Code Challenges:

1. Combine firstName and lastName LiveData
2. Create combined LiveData showing min and max of two numbers
3. Combine isLoading and hasError into single UI state

Example 20: LiveData with Default Value

```
class SettingsViewModel : ViewModel() {
    // Provide default value in constructor
    private val _isDarkMode = MutableLiveData<Boolean>(false)
    val isDarkMode: LiveData<Boolean> = _isDarkMode

    private val _fontSize = MutableLiveData<Int>(14)
    val fontSize: LiveData<Int> = _fontSize

    private val _language = MutableLiveData<String>("English")
    val language: LiveData<String> = _language

    fun toggleDarkMode() {
        // Safe to use !! here since we have default value
        _isDarkMode.value = !(_isDarkMode.value ?: false)
    }

    fun increaseFontSize() {
        val current = _fontSize.value ?: 14
        if (current < 24) {
            _fontSize.value = current + 2
        }
    }
}

// Benefits of default values:
// - Observers get immediate value when they start observing
// - No need to check for null in every operation
// - Clear intent of initial state
```

Questions:

1. How do you set a default value in LiveData?
2. What's the benefit of providing default values?
3. Can you change the default value after initialization?

Code Challenges:

1. Create LiveData with default 'Guest'
2. Create settings LiveData with multiple defaults
3. Write a function to reset all settings to defaults

Example 21: LiveData with Validation

```
class FormViewModel : ViewModel() {
    // User input
    private val _email = MutableLiveData<String>()
    private val _password = MutableLiveData<String>()

    // Validation results as LiveData
    val isEmailValid: LiveData<Boolean> = Transformations.map(_email) { email ->
        // Basic email validation
        email.isNotEmpty() && email.contains("@") && email.contains(".")
    }

    val isPasswordValid: LiveData<Boolean> = Transformations.map(_password) { pwd ->
        // Password must be at least 8 characters
        pwd.length >= 8
    }

    // Combine validations to enable/disable submit button
    val isFormValid = MediatorLiveData<Boolean>().apply {
        fun validate() {
            val emailValid = isEmailValid.value ?: false
            val passwordValid = isPasswordValid.value ?: false
            value = emailValid && passwordValid
        }

        addSource(isEmailValid) { validate() }
        addSource(isPasswordValid) { validate() }
    }

    fun setEmail(email: String) { _email.value = email }
    fun setPassword(pwd: String) { _password.value = pwd }
}
```

Questions:

1. How does this validation approach work?
2. What are the limitations of this validation?
3. How would you display validation errors to the user?

Code Challenges:

1. Add phone number validation LiveData
2. Create validation for matching password and confirm password
3. Add real-time character count for password

Example 22: LiveData with Filtering

```
data class Item(val id: Int, val name: String, val category: String)

class FilterViewModel : ViewModel() {
    // All items from data source
    private val _allItems = MutableLiveData<List<Item>>()

    // Current filter query
    private val _filterQuery = MutableLiveData<String>("")

    // Filtered results - automatically updates when either source changes
    val filteredItems: LiveData<List<Item>> = MediatorLiveData<List<Item>>().apply {
        fun filter() {
            val query = _filterQuery.value ?: ""
            val items = _allItems.value ?: emptyList()

            value = if (query.isEmpty()) {
                // No filter - show all items
                items
            } else {
                // Filter by name (case-insensitive)
                items.filter {
                    it.name.contains(query, ignoreCase = true) ||
                    it.category.contains(query, ignoreCase = true)
                }
            }
        }

        // Re-filter when items change
        addSource(_allItems) { filter() }

        // Re-filter when query changes
        addSource(_filterQuery) { filter() }
    }

    fun setFilter(query: String) {
        _filterQuery.value = query
    }

    fun loadItems(items: List<Item>) {
        _allItems.value = items
    }
}
```

Questions:

1. Why use MediatorLiveData for filtering?
2. When does the filtering operation execute?
3. How would you improve performance for large lists?

Code Challenges:

1. Add category filter in addition to search
2. Implement filter by price range
3. Add sorting (by name, by date) with filtering

Example 23: LiveData with Computed Properties

```
class CalculatorViewModel : ViewModel() {
    private val _value1 = MutableLiveData<Int>(0)
    private val _value2 = MutableLiveData<Int>(0)

    // Computed property - sum
    val sum: LiveData<Int> = MediatorLiveData<Int>().apply {
        fun calculate() {
            value = (_value1.value ?: 0) + (_value2.value ?: 0)
        }
        addSource(_value1) { calculate() }
        addSource(_value2) { calculate() }
    }

    // Computed property - product
    val product: LiveData<Int> = MediatorLiveData<Int>().apply {
        fun calculate() {
            value = (_value1.value ?: 0) * (_value2.value ?: 0)
        }
        addSource(_value1) { calculate() }
        addSource(_value2) { calculate() }
    }

    // Computed property - average
    val average: LiveData<Double> = MediatorLiveData<Double>().apply {
        fun calculate() {
            val v1 = _value1.value ?: 0
            val v2 = _value2.value ?: 0
            value = (v1 + v2) / 2.0
        }
        addSource(_value1) { calculate() }
        addSource(_value2) { calculate() }
    }

    fun setValue1(value: Int) { _value1.value = value }
    fun setValue2(value: Int) { _value2.value = value }
}
```

Questions:

1. Are computed properties calculated eagerly or lazily?
2. What happens when an observer starts observing sum?
3. How would you handle division by zero?

Code Challenges:

1. Add difference (value1 - value2) computation
2. Create safe division with null for divide by zero
3. Add percentage calculation LiveData

Example 24: LiveData with Loading State

```
class LoadingViewModel : ViewModel() {
    // Separate LiveData for loading state
    private val _isLoading = MutableLiveData<Boolean>(false)
    val isLoading: LiveData<Boolean> = _isLoading

    // Actual data
    private val _data = MutableLiveData<String>()
    val data: LiveData<String> = _data

    // Error state
    private val _error = MutableLiveData<String?>()
    val error: LiveData<String?> = _error

    fun loadData() {
        // Show loading indicator
        _isLoading.value = true
        _error.value = null

        viewModelScope.launch {
            try {
                delay(2000) // Simulate network call
                _data.value = "Loaded data"
            } catch (e: Exception) {
                _error.value = e.message
            } finally {
                // Hide loading indicator
                _isLoading.value = false
            }
        }
    }
}

// In Activity/Fragment:
// viewModel.isLoading.observe(this) { isLoading ->
//     progressBar.visibility = if (isLoading) View.VISIBLE else View.GONE
// }
//
// viewModel.error.observe(this) { error ->
//     if (error != null) showError(error)
// }
```

Questions:

1. Why have separate LiveData for loading state?
2. What happens if loadData() is called while already loading?
3. How would you handle errors in this pattern?

Code Challenges:

1. Add check to prevent multiple simultaneous loads
2. Create combined UiState instead of separate LiveData
3. Add retry functionality after error

Example 25: LiveData with Pagination

```
class PagingViewModel : ViewModel() {
    // Current page number
    private val _currentPage = MutableLiveData<Int>(1)

    // All loaded items (accumulates across pages)
    private val _items = MutableLiveData<List<String>>(emptyList())
    val items: LiveData<List<String>> = _items

    // Loading state
    private val _isLoading = MutableLiveData<Boolean>(false)
    val isLoading: LiveData<Boolean> = _isLoading

    // Has more pages to load
    private val _hasMore = MutableLiveData<Boolean>(true)
    val hasMore: LiveData<Boolean> = _hasMore

    fun loadNextPage() {
        if (_isLoading.value == true || _hasMore.value == false) {
            return // Already loading or no more pages
        }

        val page = _currentPage.value ?: 1
        _isLoading.value = true

        viewModelScope.launch {
            try {
                val newItems = fetchPage(page)

                // Append new items to existing list
                _items.value = (_items.value ?: emptyList()) + newItems

                // Check if there are more pages
                _hasMore.value = newItems.isNotEmpty()

                // Increment page counter
                _currentPage.value = page + 1
            } finally {
                _isLoading.value = false
            }
        }
    }

    private suspend fun fetchPage(page: Int): List<String> {
        delay(500) // Simulate API call
        // Return empty list after page 5 (no more data)
        return if (page <= 5) {
            List(10) { "Item ${((page - 1) * 10 + it + 1)}" }
        } else {
            emptyList()
        }
    }

    fun refresh() {
        _currentPage.value = 1
        _items.value = emptyList()
        _hasMore.value = true
        loadNextPage()
    }
}
```

Questions:

1. How does this implement pagination?
2. What's the limitation of this approach?
3. How would you implement pull-to-refresh?

Code Challenges:

1. Add error handling for page load failures

2. Implement refresh() to reload from page 1

3. Add page size configuration

Example 26: LiveData with Timestamp

```
// Wrapper class that includes timestamp
data class TimestampedData<T> (
    val data: T,
    val timestamp: Long = System.currentTimeMillis()
) {
    // Check if data is older than specified milliseconds
    fun isOlderThan(milliseconds: Long): Boolean {
        return System.currentTimeMillis() - timestamp > milliseconds
    }

    // Check if data is fresh (less than 5 minutes old)
    fun isFresh(): Boolean = !isOlderThan(5 * 60 * 1000)
}

class TimestampViewModel : ViewModel() {
    private val _data = MutableLiveData<TimestampedData<String>>()
    val data: LiveData<TimestampedData<String>> = _data

    fun updateData(value: String) {
        // Automatically adds current timestamp
        _data.value = TimestampedData(value)
    }

    fun refreshIfStale() {
        val current = _data.value
        if (current == null || current.isOlderThan(60000)) {
            // Refresh if no data or older than 1 minute
            loadFreshData()
        }
    }

    private fun loadFreshData() {
        viewModelScope.launch {
            val freshData = fetchData()
            _data.value = TimestampedData(freshData)
        }
    }

    private suspend fun fetchData(): String {
        delay(500)
        return "Fresh data"
    }
}
```

Questions:

1. Why include timestamps with data?
2. How would you check if data is stale?
3. What are use cases for timestamped data?

Code Challenges:

1. Create cache that auto-refreshes stale data
2. Add lastUpdated field to display in UI
3. Implement TTL (time-to-live) for data

Example 27: LiveData with Event Wrapper

```
// Event wrapper that handles content only once
class Event<out T>(private val content: T) {
    // Flag to track if content has been handled
    private var hasBeenHandled = false

    // Returns content only if not handled yet, then marks as handled
    fun getContentIfNotHandled(): T? {
        return if (hasBeenHandled) {
            null // Already handled
        } else {
            hasBeenHandled = true
            content
        }
    }

    // Peek at content without marking as handled
    fun peekContent(): T = content
}

class EventViewModel : ViewModel() {
    // Use Event wrapper for one-time events
    private val _snackbarMessage = MutableLiveData<Event<String>>()
    val snackbarMessage: LiveData<Event<String>> = _snackbarMessage

    private val _navigationEvent = MutableLiveData<Event<Int>>()
    val navigationEvent: LiveData<Event<Int>> = _navigationEvent

    fun showMessage(message: String) {
        _snackbarMessage.value = Event(message)
    }

    fun navigateToDetail(itemId: Int) {
        _navigationEvent.value = Event(itemId)
    }
}

// In Activity/Fragment:
// viewModel.snackbarMessage.observe(this) { event ->
//     event.getContentIfNotHandled()?.let { message ->
//         // Show snackbar only once, even after rotation
//         Snackbar.make(view, message, Snackbar.LENGTH_SHORT).show()
//     }
// }
```

Questions:

1. What problem does Event wrapper solve?
2. How is this different from SingleLiveEvent?
3. When would you use Event wrapper?

Code Challenges:

1. Create Event for button click events
2. Implement Event for error handling
3. Create navigation event with multiple parameters

Example 28: LiveData with Two-Way Data Binding

```
class TwoWayViewModel : ViewModel() {
    // MutableLiveData exposed for two-way binding
    val text = MutableLiveData<String>("")
    val email = MutableLiveData<String>("")
    val isAgreeChecked = MutableLiveData<Boolean>(false)

    // Derived properties that update automatically
    val textLength: LiveData<Int> = Transformations.map(text) {
        it.length
    }

    val isEmailValid: LiveData<Boolean> = Transformations.map(email) {
        it.contains("@")
    }

    val canSubmit: LiveData<Boolean> = MediatorLiveData<Boolean>().apply {
        fun check() {
            val valid = (isEmailValid.value == true) &&
                (isAgreeChecked.value == true)
            value = valid
        }
        addSource(isEmailValid) { check() }
        addSource(isAgreeChecked) { check() }
    }
}

// In XML layout with Data Binding:
// <layout>
//     <data>
//         <variable name="viewModel" type="...TwoWayViewModel" />
//     </data>
//     <EditText
//         android:text="@={viewModel.text}" <!-- Two-way binding with @= -->
//         ... />
//     <CheckBox
//         android:checked="@={viewModel.isAgreeChecked}"
//         ... />
// </layout>
```

Questions:

1. What is two-way data binding?
2. What does @= mean in data binding?
3. When should you avoid two-way binding?

Code Challenges:

1. Create two-way binding for age (Int) input
2. Add validation that updates in real-time
3. Create binding for RadioGroup selection

Example 29: LiveData with Retry Logic

```
class RetryViewModel : ViewModel() {
    private val _data = MutableLiveData<String>()
    val data: LiveData<String> = _data

    private val _error = MutableLiveData<String?>()
    val error: LiveData<String?> = _error

    private val _retryCount = MutableLiveData<Int>(0)
    val retryCount: LiveData<Int> = _retryCount

    fun loadData() {
        viewModelScope.launch {
            var retries = 0
            val maxRetries = 3

            while (retries < maxRetries) {
                try {
                    _retryCount.value = retries
                    val result = fetchData()

                    // Success!
                    _data.value = result
                    _error.value = null
                    break
                } catch (e: Exception) {
                    retries++

                    if (retries >= maxRetries) {
                        // All retries exhausted
                        _error.value = "Failed after $maxRetries attempts: ${e.message}"
                    } else {
                        // Wait before retrying (exponential backoff)
                        delay(1000L * retries) // 1s, 2s, 3s
                    }
                }
            }
        }
    }

    private suspend fun fetchData(): String {
        delay(1000)
        // Simulate 70% failure rate
        if (Math.random() < 0.7) {
            throw Exception("Network error")
        }
        return "Success data"
    }

    fun retry() {
        _error.value = null
        loadData()
    }
}
```

Questions:

1. How does the retry mechanism work?
2. Why use exponential backoff (delay(1000 * retries))?
3. How would you make the retry count configurable?

Code Challenges:

1. Make max retries a constructor parameter
2. Add configurable delay between retries
3. Implement immediate retry vs delayed retry options

Example 30: LiveData with Cache

```
data class CachedValue<T>(  
    val data: T,  
    val cachedAt: Long = System.currentTimeMillis()  
)  
  
class CachedViewModel : ViewModel() {  
    // In-memory cache  
    private val cache = mutableMapOf<String, CachedValue<String>>()  
    private val cacheTimeout = 5 * 60 * 1000L // 5 minutes  
  
    private val _data = MutableLiveData<String>()  
    val data: LiveData<String> = _data  
  
    private val _isLoading = MutableLiveData<Boolean>()  
    val isLoading: LiveData<Boolean> = _isLoading  
  
    fun getData(key: String, forceRefresh: Boolean = false) {  
        // Check cache first  
        val cachedValue = cache[key]  
        val isCacheValid = cachedValue != null &&  
            (System.currentTimeMillis() - cachedValue.cachedAt) < cacheTimeout  
  
        if (!forceRefresh && isCacheValid) {  
            // Return cached data immediately  
            _data.value = cachedValue!!.data  
            return  
        }  
  
        // Fetch from network  
        _isLoading.value = true  
        viewModelScope.launch {  
            try {  
                val result = fetchFromNetwork(key)  
  
                // Update cache  
                cache[key] = CachedValue(result)  
  
                // Update LiveData  
                _data.value = result  
            } catch (e: Exception) {  
                // On error, use stale cache if available  
                cachedValue?.let { _data.value = it.data }  
            } finally {  
                _isLoading.value = false  
            }  
        }  
    }  
  
    private suspend fun fetchFromNetwork(key: String): String {  
        delay(1000)  
        return "Data for $key"  
    }  
  
    fun clearCache() {  
        cache.clear()  
    }  
  
    fun invalidateKey(key: String) {  
        cache.remove(key)  
    }  
}
```

Questions:

1. What are the benefits of caching?
2. What's missing from this cache implementation?
3. How would you implement cache expiration?

Code Challenges:

1. Add maximum cache size with LRU eviction
2. Implement persistent cache using Room database
3. Add cache warming on app start

Example 31: LiveData Shared Between Fragments

```
// Shared ViewModel at Activity scope
class SharedViewModel : ViewModel() {
    private val _sharedData = MutableLiveData<String>()
    val sharedData: LiveData<String> = _sharedData

    private val _selectedItem = MutableLiveData<Item?>()
    val selectedItem: LiveData<Item?> = _selectedItem

    fun updateData(data: String) {
        _sharedData.value = data
    }

    fun selectItem(item: Item) {
        _selectedItem.value = item
    }
}

// Fragment 1 - List Fragment
class Fragment1 : Fragment() {
    // activityViewModels() gets ViewModel scoped to Activity
    private val sharedViewModel: SharedViewModel by activityViewModels()

    override fun onCreateView(view: View, savedInstanceState: Bundle?) {
        // Fragment 1 can update shared data
        button.setOnClickListener {
            sharedViewModel.updateData("Data from Fragment 1")
        }

        // Fragment 1 can also observe
        sharedViewModel.selectedItem.observe(viewLifecycleOwner) { item ->
            // React to item selected in Fragment 2
        }
    }
}

// Fragment 2 - Detail Fragment
class Fragment2 : Fragment() {
    // Gets SAME ViewModel instance as Fragment1
    private val sharedViewModel: SharedViewModel by activityViewModels()

    override fun onCreateView(view: View, savedInstanceState: Bundle?) {
        // Fragment 2 observes shared data
        sharedViewModel.sharedData.observe(viewLifecycleOwner) { data ->
            // Updates when Fragment 1 changes data
            textView.text = data
        }

        // Fragment 2 can update selected item
        itemView.setOnClickListener {
            sharedViewModel.selectItem(currentItem)
        }
    }
}
```

Questions:

1. How do multiple fragments share a ViewModel?
2. What's the difference between `viewModels()` and `activityViewModels()`?
3. When are all observers notified?

Code Challenges:

1. Create `SharedViewModel` for master-detail pattern
2. Implement fragment communication without shared `ViewModel`
3. Add third fragment that also uses `SharedViewModel`

Example 32: LiveData with Distinct Values

```
class DistinctViewModel : ViewModel() {
    private val _value = MutableLiveData<String>()

    // distinctUntilChanged() only emits when value actually changes
    // Prevents duplicate notifications with same value
    val distinctValue: LiveData<String> = _value.distinctUntilChanged()

    private val _number = MutableLiveData<Int>()

    // Custom equality check
    val evenOrOdd: LiveData<String> = Transformations.distinctUntilChanged(
        _number,
        // Only emit when even/odd state changes
        { it % 2 == 0 }
    )

    fun setValue(value: String) {
        // Even if same value is set multiple times
        _value.value = value // Observer called every time
        // distinctValue observer only called when value changes
    }

    fun setNumber(n: Int) {
        _number.value = n
    }
}

// Example:
// setValue("Hello") -> distinctValue emits "Hello"
// setValue("Hello") -> distinctValue does NOT emit
// setValue("World") -> distinctValue emits "World"
// setValue("World") -> distinctValue does NOT emit

// setNumber(2) -> evenOrOdd doesn't emit (or emits initial)
// setNumber(4) -> evenOrOdd doesn't emit (both even)
// setNumber(5) -> evenOrOdd emits (changed to odd)
// setNumber(7) -> evenOrOdd doesn't emit (both odd)
```

Questions:

1. What does `distinctUntilChanged()` do?
2. When would you use this?
3. How does it compare values for equality?

Code Challenges:

1. Create distinct LiveData for user search queries
2. Implement distinct for custom object with `equals()`
3. Use distinct to prevent redundant network calls

Example 33: LiveData Transformation Chain

```
class ChainViewModel : ViewModel() {
    // Source LiveData
    private val _input = MutableLiveData<String>()

    // Chain multiple transformations
    val output: LiveData<String> = _input
        .map { it.trim() } // Remove whitespace
        .map { it.uppercase() } // Convert to uppercase
        .map { "Processed: $it" } // Add prefix
        .distinctUntilChanged() // Remove duplicates

    // More complex chain
    private val _number = MutableLiveData<Int>()

    val processedNumber: LiveData<String> = _number
        .map { it * 2 } // Double it
        .map { if (it < 0) 0 else it } // Ensure non-negative
        .map { "$it points" } // Format as string

    // Chain with switchMap
    private val _userId = MutableLiveData<Int>()

    val userDetails: LiveData<String> = _userId
        .switchMap { id -> getUserLiveData(id) }
        .map { user -> "${user.name} (${user.email})" }
        .distinctUntilChanged()

    fun setInput(value: String) {
        _input.value = value
    }

    fun setNumber(n: Int) {
        _number.value = n
    }

    fun setUserId(id: Int) {
        _userId.value = id
    }

    private fun getUserLiveData(id: Int): LiveData<User> {
        val result = MutableLiveData<User>()
        result.value = User(id, "User$id", "user$id@example.com")
        return result
    }
}

data class User(val id: Int, val name: String, val email: String)
```

Questions:

1. How many transformations are applied to output?
2. What order do the transformations execute?
3. Can you chain other operations besides map()?

Code Challenges:

1. Add validation step to the chain
2. Create chain that filters then transforms
3. Build chain with error handling at each step

Example 34: Shopping Cart with LiveData

```
data class Product(  
    val id: Int,  
    val name: String,  
    val price: Double,  
    val quantity: Int = 1  
)  
  
class CartViewModel : ViewModel() {  
    // Cart items  
    private val _cart = MutableLiveData<List<Product>>(emptyList())  
    val cart: LiveData<List<Product>> = _cart  
  
    // Computed: Total price  
    val totalPrice: LiveData<Double> = Transformations.map(_cart) { products ->  
        products.sumOf { it.price * it.quantity }  
    }  
  
    // Computed: Item count  
    val itemCount: LiveData<Int> = Transformations.map(_cart) { products ->  
        products.sumOf { it.quantity }  
    }  
  
    // Computed: Can checkout  
    val canCheckout: LiveData<Boolean> = Transformations.map(_cart) {  
        it.isNotEmpty()  
    }  
  
    fun addProduct(product: Product) {  
        val currentCart = _cart.value ?: emptyList()  
  
        // Check if product already in cart  
        val existingIndex = currentCart.indexOfFirst { it.id == product.id }  
  
        _cart.value = if (existingIndex >= 0) {  
            // Increase quantity of existing product  
            currentCart.toMutableList().apply {  
                val existing = get(existingIndex)  
                set(existingIndex, existing.copy(quantity = existing.quantity + 1))  
            }  
        } else {  
            // Add new product  
            currentCart + product  
        }  
    }  
  
    fun removeProduct(productId: Int) {  
        val currentCart = _cart.value ?: emptyList()  
        _cart.value = currentCart.filter { it.id != productId }  
    }  
  
    fun updateQuantity(productId: Int, newQuantity: Int) {  
        val currentCart = _cart.value ?: emptyList()  
        _cart.value = currentCart.map { product ->  
            if (product.id == productId) {  
                product.copy(quantity = newQuantity)  
            } else {  
                product  
            }  
        }  
    }  
  
    fun clearCart() {  
        _cart.value = emptyList()  
    }  
}
```

Questions:

1. Why calculate totalPrice as LiveData instead of a function?
2. How would you handle product quantity?
3. What happens when cart changes?

Code Challenges:

1. Add discount code functionality
2. Implement 'save for later' feature
3. Add shipping cost calculation

Example 35: LiveData with Room Database

```
// Room Entity
@Entity(tableName = "users")
data class User(
    @PrimaryKey val id: Int,
    val name: String,
    val email: String,
    val age: Int
)

// Room DAO - LiveData is automatically updated when DB changes
@Dao
interface UserDao {
    // Query returns LiveData - automatically updates
    @Query("SELECT * FROM users")
    fun getAllUsers(): LiveData<List<User>>

    @Query("SELECT * FROM users WHERE id = :userId")
    fun getUserById(userId: Int): LiveData<User>

    @Query("SELECT * FROM users WHERE age >= :minAge")
    fun getUsersAboveAge(minAge: Int): LiveData<List<User>>

    // These operations run on background thread
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertUser(user: User)

    @Update
    suspend fun updateUser(user: User)

    @Delete
    suspend fun deleteUser(user: User)
}

class DatabaseViewModel(private val userDao: UserDao) : ViewModel() {
    // Room automatically updates this LiveData when database changes
    val allUsers: LiveData<List<User>> = userDao.getAllUsers()

    fun getUser(id: Int): LiveData<User> = userDao.getUserById(id)

    fun getUsersAboveAge(age: Int): LiveData<List<User>> =
        userDao.getUsersAboveAge(age)

    // Modify database
    fun addUser(user: User) {
        viewModelScope.launch {
            userDao.insertUser(user)
            // allUsers automatically updates!
        }
    }

    fun updateUser(user: User) {
        viewModelScope.launch {
            userDao.updateUser(user)
        }
    }

    fun deleteUser(user: User) {
        viewModelScope.launch {
            userDao.deleteUser(user)
        }
    }
}
```

Questions:

1. How does Room provide LiveData?
2. When does the database query execute?
3. What thread does Room use for queries?

Code Challenges:

1. Add search query with LiveData return type
2. Create complex query joining multiple tables
3. Implement sorting options with LiveData

Example 36: Converting Flow to LiveData

```
class FlowViewModel : ViewModel() {
    // Flow - newer alternative to LiveData
    private val _flow = MutableStateFlow("Initial")

    // Convert Flow to LiveData for use with older code
    val liveData: LiveData<String> = _flow.asLiveData()

    // SharedFlow to LiveData
    private val _sharedFlow = MutableSharedFlow<String>()
    val sharedLiveData: LiveData<String> = _sharedFlow.asLiveData()

    // Flow with transformations
    val transformedLiveData: LiveData<String> = _flow
        .map { it.uppercase() }
        .distinctUntilChanged()
        .asLiveData()

    fun updateValue(value: String) {
        _flow.value = value
    }

    fun emitEvent(event: String) {
        viewModelScope.launch {
            _sharedFlow.emit(event)
        }
    }
}

// When to use what:
// - Flow: Better for streams, hot/cold data, complex operators
// - LiveData: Better for UI-related data, lifecycle awareness built-in
// - Use asLiveData() to bridge between them
```

Questions:

1. What is Flow and how does it differ from LiveData?
2. Why convert Flow to LiveData?
3. When should you use Flow vs LiveData?

Code Challenges:

1. Convert Room Flow query to LiveData
2. Create Flow that emits every second, convert to LiveData
3. Use asLiveData with specific coroutine context

Example 37: LiveData with Error Handling

```
// Result wrapper for success/error states
sealed class Result<out T> {
    data class Success<T>(val data: T) : Result<T>()
    data class Error(val exception: Exception) : Result<Nothing>()
    data class Loading(val message: String = "Loading...") : Result<Nothing>()
}

class SafeViewModel : ViewModel() {
    private val _result = MutableLiveData<Result<String>>()
    val result: LiveData<Result<String>> = _result

    fun loadData() {
        // Set loading state
        _result.value = Result.Loading()

        viewModelScope.launch {
            try {
                val data = fetchData()
                _result.value = Result.Success(data)
            } catch (e: Exception) {
                _result.value = Result.Error(e)
            }
        }
    }

    private suspend fun fetchData(): String {
        delay(1000)
        // Simulate random error
        if (Math.random() < 0.3) {
            throw Exception("Network error")
        }
        return "Success data"
    }
}

// In Activity/Fragment - Handle all cases:
// viewModel.result.observe(this) { result ->
//     when (result) {
//         is Result.Loading -> {
//             progressBar.visibility = View.VISIBLE
//             errorView.visibility = View.GONE
//             contentView.visibility = View.GONE
//         }
//         is Result.Success -> {
//             progressBar.visibility = View.GONE
//             errorView.visibility = View.GONE
//             contentView.visibility = View.VISIBLE
//             textView.text = result.data
//         }
//         is Result.Error -> {
//             progressBar.visibility = View.GONE
//             errorView.visibility = View.VISIBLE
//             contentView.visibility = View.GONE
//             errorText.text = result.exception.message
//         }
//     }
// }
```

Questions:

1. What are the benefits of wrapping results?
2. How would you display errors in the UI?
3. What's another way to handle errors with LiveData?

Code Challenges:

1. Add retry mechanism to error handling
2. Create Result type with partial data in error state

3. Implement exponential backoff with Result

Example 38: LiveData with Dependency Injection

```
// Repository interface for abstraction
interface NetworkRepository {
    fun getData(): LiveData<String>
    suspend fun refreshData()
}

// Concrete implementation
class NetworkRepositoryImpl(
    private val apiService: ApiService
) : NetworkRepository {
    private val _data = MutableLiveData<String>()

    override fun getData(): LiveData<String> = _data

    override suspend fun refreshData() {
        val result = apiService.fetchData()
        _data.postValue(result)
    }
}

// ViewModel with constructor injection
class InjectedViewModel(
    private val repository: NetworkRepository
) : ViewModel() {
    // Get data from repository
    val data: LiveData<String> = repository.getData()

    fun refresh() {
        viewModelScope.launch {
            repository.refreshData()
        }
    }
}

// ViewModelFactory for manual injection
class ViewModelFactory(
    private val repository: NetworkRepository
) : ViewModelProvider.Factory {
    override fun <T : ViewModel> create(modelClass: Class<T>): T {
        if (modelClass.isAssignableFrom(InjectedViewModel::class.java)) {
            @Suppress("UNCHECKED_CAST")
            return InjectedViewModel(repository) as T
        }
        throw IllegalArgumentException("Unknown ViewModel class")
    }
}

// Usage in Activity:
// val repository = NetworkRepositoryImpl(apiService)
// val factory = ViewModelFactory(repository)
// val viewModel = ViewModelProvider(this, factory)
// .get(InjectedViewModel::class.java)

// With Hilt:
// @HiltViewModel
// class InjectedViewModel @Inject constructor(
//     private val repository: NetworkRepository
// ) : ViewModel() { ... }
```

Questions:

1. Why use dependency injection?
2. What is `ViewModelProvider.Factory`?
3. How would you use Hilt or Dagger for injection?

Code Challenges:

1. Create `ViewModel` with multiple repository dependencies
2. Write test using mock repository

3. Implement Hilt injection for ViewModel

Example 39: LiveData with Polling

```
class PollingViewModel : ViewModel() {
    private val _data = MutableLiveData<String>()
    val data: LiveData<String> = _data

    private val _lastUpdate = MutableLiveData<Long>()
    val lastUpdate: LiveData<Long> = _lastUpdate

    // Store job reference to control polling
    private var pollingJob: Job? = null

    fun startPolling(intervalMs: Long = 5000) {
        // Stop existing polling if any
        stopPolling()

        pollingJob = viewModelScope.launch {
            // Infinite loop while coroutine is active
            while (isActive) {
                try {
                    // Fetch fresh data
                    val freshData = fetchData()
                    _data.value = freshData
                    _lastUpdate.value = System.currentTimeMillis()
                } catch (e: Exception) {
                    // Log error but continue polling
                    Log.e("PollingViewModel", "Poll failed", e)
                }

                // Wait before next poll
                delay(intervalMs)
            }
        }
    }

    fun stopPolling() {
        // Cancel the polling coroutine
        pollingJob?.cancel()
        pollingJob = null
    }

    private suspend fun fetchData(): String {
        delay(100)
        return "Data at ${System.currentTimeMillis()}"
    }

    override fun onCleared() {
        super.onCleared()
        // Clean up when ViewModel is destroyed
        stopPolling()
    }
}

// Usage in Activity/Fragment:
// override fun onResume() {
//     super.onResume()
//     viewModel.startPolling(3000) // Poll every 3 seconds
// }
//
// override fun onPause() {
//     super.onPause()
//     viewModel.stopPolling() // Stop when not visible
// }
```

Questions:

1. What is polling and when would you use it?
2. Why store the Job reference?
3. How would you handle errors during polling?

Code Challenges:

1. Add configurable retry count for failed polls
2. Implement adaptive polling (faster when active, slower when idle)
3. Create polling that stops after N successful fetches

Example 40: LiveData with Conditional Updates

```
class ConditionalViewModel : ViewModel() {
    // Counter with max value constraint
    private val _value = MutableLiveData<Int>(0)
    val value: LiveData<Int> = _value

    // Validation errors
    private val _error = MutableLiveData<String?>()
    val error: LiveData<String?> = _error

    fun increment() {
        val current = _value.value ?: 0

        if (current < 100) {
            _value.value = current + 1
            _error.value = null
        } else {
            _error.value = "Maximum value reached"
        }
    }

    fun decrement() {
        val current = _value.value ?: 0

        if (current > 0) {
            _value.value = current - 1
            _error.value = null
        } else {
            _error.value = "Minimum value reached"
        }
    }

    fun setValue(newValue: Int) {
        when {
            newValue < 0 -> {
                _error.value = "Value cannot be negative"
            }
            newValue > 100 -> {
                _error.value = "Value cannot exceed 100"
            }
            else -> {
                _value.value = newValue
                _error.value = null
            }
        }
    }

    // Only update if different from current
    fun setValueIfDifferent(newValue: Int) {
        if (_value.value != newValue) {
            setValue(newValue)
        }
    }
}
```

Questions:

1. Why validate before updating LiveData?
2. Should validation be in ViewModel or elsewhere?
3. How would you notify users of invalid input?

Code Challenges:

1. Add custom validation rule as parameter
2. Create conditional update for email format
3. Implement min/max constraints as constructor params

Example 41: LiveData with Resource Pattern

```
// Resource pattern includes data at all states
sealed class Resource<T>{
    val data: T? = null,
    val message: String? = null
} {
    // Success with data
    class Success<T>(data: T) : Resource<T>(data)

    // Error with optional stale data
    class Error<T>(message: String, data: T? = null) : Resource<T>(data, message)

    // Loading with optional cached data
    class Loading<T>(data: T? = null) : Resource<T>(data)
}

class ResourceViewModel : ViewModel() {
    private val _resource = MutableLiveData<Resource<List<String>>>()
    val resource: LiveData<Resource<List<String>>> = _resource

    fun loadData(useCache: Boolean = true) {
        // Start with loading, include cached data if available
        val cachedData = if (useCache) _resource.value?.data else null
        _resource.value = Resource.Loading(cachedData)

        viewModelScope.launch {
            try {
                delay(1000)
                val freshData = listOf("Item 1", "Item 2", "Item 3")
                _resource.value = Resource.Success(freshData)
            } catch (e: Exception) {
                // Include stale data in error state
                _resource.value = Resource.Error(
                    e.message ?: "Unknown error",
                    cachedData
                )
            }
        }
    }
}

// In Activity - Show data even during loading/error:
// viewModel.resource.observe(this) { resource ->
//     when (resource) {
//         is Resource.Loading -> {
//             progressBar.visibility = View.VISIBLE
//             // Still show cached data if available
//             resource.data?.let { showData(it) }
//         }
//         is Resource.Success -> {
//             progressBar.visibility = View.GONE
//             showData(resource.data)
//         }
//         is Resource.Error -> {
//             progressBar.visibility = View.GONE
//             showError(resource.message)
//             // Show stale data if available
//             resource.data?.let { showData(it) }
//         }
//     }
// }
```

Questions:

1. What's the difference between Resource and Result patterns?
2. Why can Loading include data?
3. How would you show a progress bar with this pattern?

Code Challenges:

1. Add Empty state to Resource

2. Create Resource with progress percentage
3. Implement cache-then-network strategy

Example 42: LiveData with SharedPreferences

```
class PreferencesViewModel(
    private val preferences: SharedPreferences
) : ViewModel() {
    // LiveData for dark mode setting
    private val _darkMode = MutableLiveData<Boolean>()
    val darkMode: LiveData<Boolean> = _darkMode

    private val _fontSize = MutableLiveData<Int>()
    val fontSize: LiveData<Int> = _fontSize

    // Listener for preference changes
    private val prefListener = SharedPreferences.OnSharedPreferenceChangeListener {
        prefs, key ->
        when (key) {
            "dark_mode" -> _darkMode.value = prefs.getBoolean(key, false)
            "font_size" -> _fontSize.value = prefs.getInt(key, 14)
        }
    }

    init {
        // Register listener
        preferences.registerOnSharedPreferenceChangeListener(prefListener)

        // Load initial values
        _darkMode.value = preferences.getBoolean("dark_mode", false)
        _fontSize.value = preferences.getInt("font_size", 14)
    }

    fun toggleDarkMode() {
        val newValue = !(_darkMode.value ?: false)
        preferences.edit().putBoolean("dark_mode", newValue).apply()
        _darkMode.value = newValue
    }

    fun setFontSize(size: Int) {
        preferences.edit().putInt("font_size", size).apply()
        _fontSize.value = size
    }

    override fun onCleared() {
        super.onCleared()
        // Unregister listener to prevent leaks
        preferences.unregisterOnSharedPreferenceChangeListener(prefListener)
    }
}

// Better alternative - Use DataStore:
// val darkModeFlow: Flow<Boolean> = datastore.data
//     .map { it[DARK_MODE_KEY] ?: false }
// val darkMode = darkModeFlow.asLiveData()
```

Questions:

1. How would you make preferences updates reactive?
2. What's better than SharedPreferences for this?
3. Should preferences logic be in ViewModel?

Code Challenges:

1. Add multiple preference settings
2. Implement preferences reset to defaults
3. Convert to DataStore implementation

Example 43: LiveData with Timeout

```
class TimeoutViewModel : ViewModel() {
    private val _data = MutableLiveData<String>()
    val data: LiveData<String> = _data

    private val _timedOut = MutableLiveData<Boolean>(false)
    val timedOut: LiveData<Boolean> = _timedOut

    fun loadWithTimeout(timeoutMs: Long = 5000) {
        _timedOut.value = false

        viewModelScope.launch {
            try {
                // withTimeout throws TimeoutCancellationException
                withTimeout(timeoutMs) {
                    val result = fetchData()
                    _data.value = result
                }
            } catch (e: TimeoutCancellationException) {
                _timedOut.value = true
                _data.value = "Request timed out after ${timeoutMs}ms"
            } catch (e: Exception) {
                _data.value = "Error: ${e.message}"
            }
        }
    }

    // Alternative: withTimeoutOrNull returns null instead of throwing
    fun loadWithTimeoutOrNull(timeoutMs: Long = 5000) {
        viewModelScope.launch {
            val result = withTimeoutOrNull(timeoutMs) {
                fetchData()
            }

            if (result != null) {
                _data.value = result
                _timedOut.value = false
            } else {
                _data.value = "Request timed out"
                _timedOut.value = true
            }
        }
    }

    private suspend fun fetchData(): String {
        // Simulate slow network
        delay(3000)
        return "Success"
    }
}
```

Questions:

1. What does withTimeout do?
2. What happens if the operation takes longer than timeout?
3. How would you configure timeout duration?

Code Challenges:

1. Add retry after timeout
2. Implement increasing timeout on each retry
3. Create configurable timeout as constructor param

Example 44: LiveData with State Machine

```
// Define all possible states
sealed class ViewState {
    object Idle : ViewState()
    object Loading : ViewState()
    data class Content(val data: String) : ViewState()
    data class Error(val message: String) : ViewState()
    object Empty : ViewState()
}

class StateMachineViewModel : ViewModel() {
    private val _state = MutableLiveData<ViewState>(ViewState.Idle)
    val state: LiveData<ViewState> = _state

    fun load() {
        // Prevent multiple simultaneous loads
        if (_state.value is ViewState.Loading) return

        _state.value = ViewState.Loading

        viewModelScope.launch {
            try {
                delay(2000)
                val data = fetchData()

                _state.value = if (data.isEmpty()) {
                    ViewState.Empty
                } else {
                    ViewState.Content(data)
                }
            } catch (e: Exception) {
                _state.value = ViewState.Error(e.message ?: "Unknown error")
            }
        }
    }

    fun reset() {
        _state.value = ViewState.Idle
    }

    fun retry() {
        if (_state.value is ViewState.Error) {
            load()
        }
    }

    private suspend fun fetchData(): String {
        delay(1000)
        return "Loaded data"
    }
}

// State machine transitions:
// Idle -> Loading (when load() called)
// Loading -> Content (on success with data)
// Loading -> Empty (on success with no data)
// Loading -> Error (on failure)
// Error -> Loading (when retry() called)
// * -> Idle (when reset() called)
```

Questions:

1. What is a state machine?
2. Why check current state before loading?
3. What transitions are valid from Loading state?

Code Challenges:

1. Add Refreshing state separate from Loading
2. Implement state history to enable undo

3. Create state machine diagram for authentication

Example 45: LiveData with Batch Updates

```
class BatchViewModel : ViewModel() {
    private val _items = MutableLiveData<List<String>>(emptyList())
    val items: LiveData<List<String>> = _items

    // Add single item
    fun addItem(item: String) {
        val current = _items.value ?: emptyList()
        _items.value = current + item
    }

    // Add multiple items in one update
    fun addItems(newItems: List<String>) {
        val current = _items.value ?: emptyList()
        // Single LiveData update for all items
        _items.value = current + newItems
    }

    // Remove multiple items at once
    fun removeItems(itemsToRemove: List<String>) {
        val current = _items.value ?: emptyList()
        // Single update removes all specified items
        _items.value = current.filter { it !in itemsToRemove }
    }

    // Bulk operation - modify multiple items
    fun updateItems(transform: (String) -> String) {
        val current = _items.value ?: emptyList()
        // Apply transformation to all items, single update
        _items.value = current.map(transform)
    }

    // Replace entire list
    fun replaceAll(newItems: List<String>) {
        _items.value = newItems
    }

    // Batch update with transaction-like semantics
    fun batchUpdate(operations: BatchViewModel.() -> Unit) {
        // Temporarily prevent updates
        val originalValue = _items.value

        try {
            operations()
        } catch (e: Exception) {
            // Rollback on error
            _items.value = originalValue
            throw e
        }
    }
}

// Usage:
// Add 100 items efficiently:
// viewModel.addItems(List(100) { "Item $it" }) // One update
//
// vs inefficient:
// for (i in 0..99) {
//     viewModel.addItem("Item $i") // 100 updates!
// }
```

Questions:

1. Why batch updates instead of one at a time?
2. What's the performance benefit?
3. How would you animate batch changes?

Code Challenges:

1. Implement batch delete by indices

2. Add batch update with progress callback
3. Create batch operation with validation

Example 46: LiveData Testing

```
class TestableViewModel : ViewModel() {
    private val _data = MutableLiveData<String>()
    val data: LiveData<String> = _data

    private val _count = MutableLiveData<Int>(0)
    val count: LiveData<Int> = _count

    fun loadData() {
        _data.value = "Test data"
    }

    fun increment() {
        _count.value = (_count.value ?: 0) + 1
    }
}

// Test class
class TestableViewModelTest {
    // Rule to execute LiveData operations synchronously
    @get:Rule
    val instantExecutorRule = InstantTaskExecutorRule()

    private lateinit var viewModel: TestableViewModel

    @Before
    fun setup() {
        viewModel = TestableViewModel()
    }

    @Test
    fun testLoadData() {
        // Observe LiveData
        val observer = Observer<String> {}
        viewModel.data.observeForever(observer)

        // Trigger action
        viewModel.loadData()

        // Assert
        assertEquals("Test data", viewModel.data.value)

        // Clean up
        viewModel.data.removeObserver(observer)
    }

    @Test
    fun testIncrement() {
        val observer = Observer<Int> {}
        viewModel.count.observeForever(observer)

        // Initial value
        assertEquals(0, viewModel.count.value)

        // After increment
        viewModel.increment()
        assertEquals(1, viewModel.count.value)

        viewModel.count.removeObserver(observer)
    }
}

// Better way - Using helper extension:
fun <T> LiveData<T>.getOrAwaitValue(): T {
    var data: T? = null
    val latch = CountDownLatch(1)
    val observer = object : Observer<T> {
        override fun onChanged(value: T) {
            data = value
            latch.countDown()
            this@getOrAwaitValue.removeObserver(this)
        }
    }
    this.observeForever(observer)
    latch.await(2, TimeUnit.SECONDS)
    return data as T
}
```

```
}  
  
// Usage:  
// @Test  
// fun testWithHelper() {  
//     viewModel.loadData()  
//     assertEquals("Test data", viewModel.data.getOrAwaitValue())  
// }
```

Questions:

1. What is `InstantTaskExecutorRule`?
2. Why do we need `observeForever` in tests?
3. What's a better way to test `LiveData`?

Code Challenges:

1. Write test for error handling in `ViewModel`
2. Test `LiveData` transformation chain
3. Create test for `MediatorLiveData` combination

Example 47: LiveData Extension Functions

```
// Extension: Observe only once
fun <T> LiveData<T>.observeOnce(owner: LifecycleOwner, observer: (T) -> Unit) {
    observe(owner) { value ->
        observer(value)
        removeObservers(owner)
    }
}

// Extension: Get non-null value or throw
fun <T> LiveData<T>.requireValue(): T {
    return this.value ?: throw IllegalStateException("LiveData value is null")
}

// Extension: Map with null check
fun <T, R> LiveData<T>.mapNotNull(transform: (T) -> R?): LiveData<R> {
    val result = MediatorLiveData<R>()
    result.addSource(this) { value ->
        val transformed = transform(value)
        if (transformed != null) {
            result.value = transformed
        }
    }
    return result
}

// Extension: Combine two LiveData
fun <A, B, R> LiveData<A>.combineWith(
    other: LiveData<B>,
    combiner: (A?, B?) -> R
): LiveData<R> {
    val result = MediatorLiveData<R>()
    result.addSource(this) { a ->
        result.value = combiner(a, other.value)
    }
    result.addSource(other) { b ->
        result.value = combiner(this.value, b)
    }
    return result
}

class ExtensionsViewModel : ViewModel() {
    private val _event = MutableLiveData<String>()
    val event: LiveData<String> = _event

    private val _firstName = MutableLiveData<String>()
    private val _lastName = MutableLiveData<String>()

    // Use extension to combine
    val fullName = _firstName.combineWith(_lastName) { first, last ->
        "${first ?: ""} ${last ?: ""}".trim()
    }

    fun triggerEvent() {
        _event.value = "Event triggered"
    }
}

// Usage:
// viewModel.event.observeOnce(this) { event ->
//     // Handle one-time event
//     showToast(event)
// }
```

Questions:

1. What does `observeOnce()` do?
2. When would you use extension functions?
3. What's the risk of `requireValue()`?

Code Challenges:

1. Create extension to filter LiveData by predicate
2. Write extension for debouncing LiveData updates
3. Implement extension to merge multiple LiveData sources

Example 48: LiveData with WorkManager

```
class WorkViewModel(application: Application) : AndroidViewModel(application) {
    private val workManager = WorkManager.getInstance(application)

    // Schedule one-time work and observe its status
    fun scheduleWork(): LiveData<WorkInfo> {
        val request = OneTimeWorkRequestBuilder<MyWorker>()
            .setInputData(workDataOf("key" to "value"))
            .build()

        workManager.enqueue(request)

        // WorkManager provides LiveData for work status
        return workManager.getWorkInfoByIdLiveData(request.id)
    }

    // Schedule periodic work
    fun schedulePeriodicWork(): LiveData<WorkInfo> {
        val request = PeriodicWorkRequestBuilder<MyWorker>(
            15, TimeUnit.MINUTES // Minimum: 15 minutes
        ).build()

        workManager.enqueue(request)
        return workManager.getWorkInfoByIdLiveData(request.id)
    }

    // Observe all work with a tag
    fun observeWorkByTag(tag: String): LiveData<List<WorkInfo>> {
        return workManager.getWorkInfosByTagLiveData(tag)
    }
}

// Worker implementation
class MyWorker(
    context: Context,
    params: WorkerParameters
) : Worker(context, params) {
    override fun doWork(): Result {
        // Do background work here
        val input = inputData.getString("key")

        // Simulate work
        Thread.sleep(5000)

        return Result.success(
            workDataOf("result" to "Work completed")
        )
    }
}

// Observing work status:
// viewModel.scheduleWork().observe(this) { workInfo ->
//     when (workInfo.state) {
//         // WorkInfo.State.ENQUEUED -> showStatus("Queued")
//         // WorkInfo.State.RUNNING -> showStatus("Running")
//         // WorkInfo.State.SUCCEEDED -> {
//         //     val result = workInfo.outputData.getString("result")
//         //     showStatus("Success: $result")
//         // }
//         // WorkInfo.State.FAILED -> showStatus("Failed")
//         // WorkInfo.State.CANCELLED -> showStatus("Cancelled")
//         // else -> {}
//     }
// }
```

Questions:

1. How does WorkManager integrate with LiveData?
2. What is WorkInfo?
3. When would you use WorkManager vs coroutines?

Code Challenges:

1. Create work chain with dependencies
2. Add constraints (network, charging) to work
3. Implement retry logic for failed work

Example 49: LiveData with Navigation

```
// Event wrapper (from Example 27)
class Event<out T>(private val content: T) {
    private var hasBeenHandled = false

    fun getContentIfNotHandled(): T? {
        return if (hasBeenHandled) null
        else {
            hasBeenHandled = true
            content
        }
    }
}

// Navigation event data
data class NavigationEvent(
    val destinationId: Int,
    val args: Bundle? = null
)

class NavigationViewModel : ViewModel() {
    // Navigation events
    private val _navigationEvent = MutableLiveData<Event<NavigationEvent>>()
    val navigationEvent: LiveData<Event<NavigationEvent>> = _navigationEvent

    // Back navigation
    private val _navigateBack = MutableLiveData<Event<Unit>>()
    val navigateBack: LiveData<Event<Unit>> = _navigateBack

    fun navigateToDetail(itemId: Int) {
        val args = Bundle().apply {
            putInt("itemId", itemId)
        }
        _navigationEvent.value = Event(
            NavigationEvent(R.id.action_list_to_detail, args)
        )
    }

    fun navigateToSettings() {
        _navigationEvent.value = Event(
            NavigationEvent(R.id.action_to_settings)
        )
    }

    fun goBack() {
        _navigateBack.value = Event(Unit)
    }
}

// In Fragment:
// viewModel.navigationEvent.observe(viewLifecycleOwner) { event ->
//     event.getContentIfNotHandled()?.let { navEvent ->
//         findNavController().navigate(
//             navEvent.destinationId,
//             navEvent.args
//         )
//     }
// }
//
// viewModel.navigateBack.observe(viewLifecycleOwner) { event ->
//     event.getContentIfNotHandled()?.let {
//         findNavController().navigateUp()
//     }
// }
```

Questions:

1. Why use Event wrapper for navigation?
2. What problem does this solve?
3. How would you pass complex data to destination?

Code Challenges:

1. Add navigation with animation options
2. Implement deep link navigation with LiveData
3. Create navigation graph with multiple paths

Example 50: LiveData Best Practices Summary

```
// Summary of all best practices in one ViewModel
class BestPracticesViewModel : ViewModel() {

    // 1. Always expose immutable LiveData publicly
    private val _data = MutableLiveData<String>()
    val data: LiveData<String> = _data

    // 2. Use meaningful names
    private val _isLoading = MutableLiveData<Boolean>(false)
    val isLoading: LiveData<Boolean> = _isLoading

    // 3. Provide default values when appropriate
    private val _count = MutableLiveData<Int>(0)
    val count: LiveData<Int> = _count

    // 4. Use transformations for derived data
    val doubleCount: LiveData<Int> = Transformations.map(_count) { it * 2 }

    // 5. Use sealed classes for complex states
    private val _uiState = MutableLiveData<UiState>()
    val uiState: LiveData<UiState> = _uiState

    // 6. Handle null safely with elvis operator
    fun increment() {
        _count.value = (_count.value ?: 0) + 1
    }

    // 7. Use viewModelScope for coroutines
    fun loadData() {
        _isLoading.value = true
        viewModelScope.launch {
            try {
                _data.value = fetchData()
            } catch (e: Exception) {
                // Handle error
            } finally {
                _isLoading.value = false
            }
        }
    }

    // 8. Use postValue() for background threads
    fun loadDataBackground() {
        viewModelScope.launch(Dispatchers.IO) {
            val result = fetchData()
            _data.postValue(result) // Thread-safe
        }
    }

    // 9. Don't expose MutableLiveData
    // BAD: val data = MutableLiveData<String>()
    // GOOD: private val _data + public val data

    // 10. Use SingleLiveEvent or Event wrapper for one-time events
    private val _event = MutableLiveData<Event<String>>()
    val event: LiveData<Event<String>> = _event

    private suspend fun fetchData(): String {
        delay(1000)
        return "Data"
    }
}

sealed class UiState {
    object Loading : UiState()
    data class Success(val data: String) : UiState()
    data class Error(val message: String) : UiState()
}

// AVOID These Common Mistakes:
// ■ Exposing MutableLiveData
// ■ Using setValue() from background thread
// ■ Not handling null values
// ■ Observing LiveData with wrong lifecycle owner in Fragments
// ■ Creating memory leaks with observeForever
```

```
// ■ Not using sealed classes for complex states  
// ■ Doing heavy operations in ViewModel init block  
// ■ Storing Context reference in ViewModel  
// ■ Using LiveData for navigation events without Event wrapper
```

Questions:

1. What are the main LiveData best practices?
2. Why is each practice important?
3. What should you avoid when using LiveData?

Code Challenges:

1. Refactor bad LiveData code to follow best practices
2. Create checklist for LiveData code review
3. Write guidelines document for team LiveData usage